数据结构

在工作过程中，常见的数据结构是： 线性表、栈、队列、哈希表和二叉搜索树
线性表又分为： 数组和链表

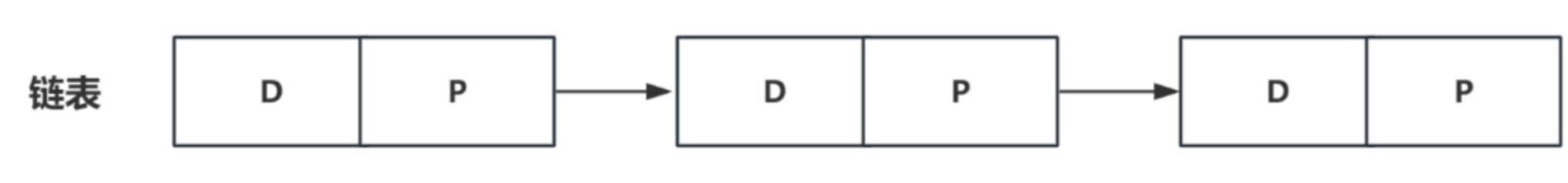
线性表是什么?
线性表，就是线型的表，数据和数据是一个挨着一个的。我们可以用一条线将所有的数据连接起来。
线性表是一个描述逻辑结构的，数组和链表是两种实现。数组是数据全部在一起，内存在一堆。链表是数据和数据不放在一起

链表中，是由一个一个的结点组成的。结点里有两个内容，一个是数据域，一个是指针域。

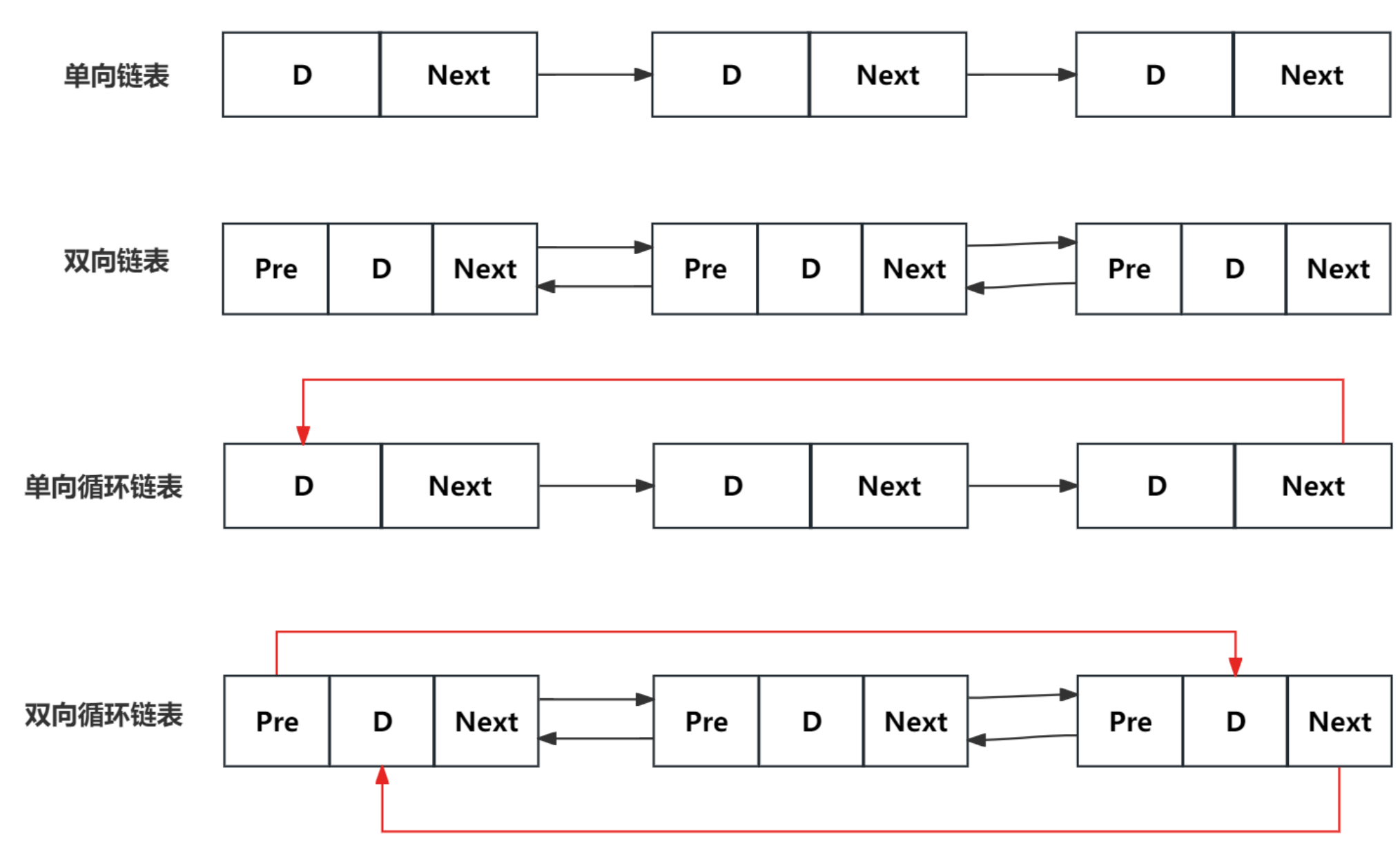
面试官问：简单聊聊数组。
简单聊聊链表。
简单聊聊数组和链表

链表

链表：是一种线性数据结构，由一系列结点"链接"构成。
结点：在C语言中，一个结点通常是一个结构体对象。该结构体对象由数据域和指针域两部分组成。数据域用于存储数据，而指针域则用于存放其它结点的指针。



常见的链表分为以下几种类型:
单向链表、双向链表、单向循环链表、双向循环链表


单向链表
D
Next
双向链表
Pre
D
Next
单向循环链表
双向循环链表

重要概念:
1.结点： 结点就是一个结构体对象，一般包括数据和指针，也就是指向下一个的指针
2.头结点：头结点有两种说法，一种是链表的第一个结点。一种是一个特殊的哑结点。这种哑结点，可以帮忙统一一些操作。

3.头指针：就是指向头结点的指针。 头指针很重要，因为基本上所有的操作都是以头指针作为入口的。

4.尾结点： 最后的那个结点。 尾结点的指针域一定是NULL。
malloc或者calloc之后。 一定要注意一下这个next的修改。

5.尾指针： 指针，这个指针指向尾结点。

链表的基本结构：链表是由结点构成，结点里有数据域，有指针域。
在C语言中，我们实现链表，需要依靠结构体。

```
typedef  int DataType;

typedef struct node_s{
  DataType data;
  struct node_s *next;
}Node;

typedef struct {
   Node *head;
   Node *tail;
   int size;
}LinkedList;
```

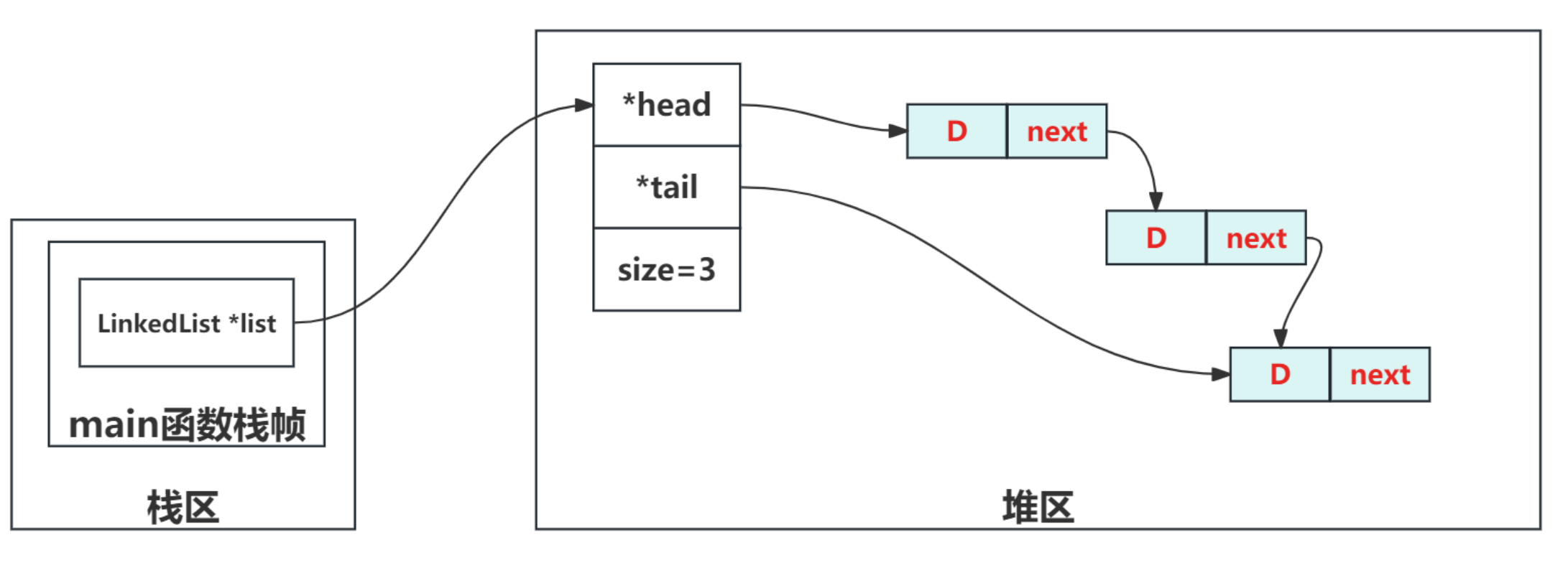
主体功能(重要功能)

1.创建单链表
2.销毁单链表
3.打印单链表
4.头插数据
5.尾插数据
6.在某个位置插入(idx)

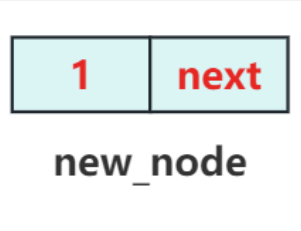
7.根据值来删除节点
8.根据位置来删除
9.改某个位置的数据，到新值
10.给数据你，查是否存在
11.给数据你，返回指针
12.给数据你，返回下标我

增删改查

这些通用的数据结构，我们肯定希望别人更容易调用。
所以我们还是采用头文件+实现的组织形式。


LinkedList *list
main函数栈帧
栈区
*head
*tail
size=3
D
next
堆区

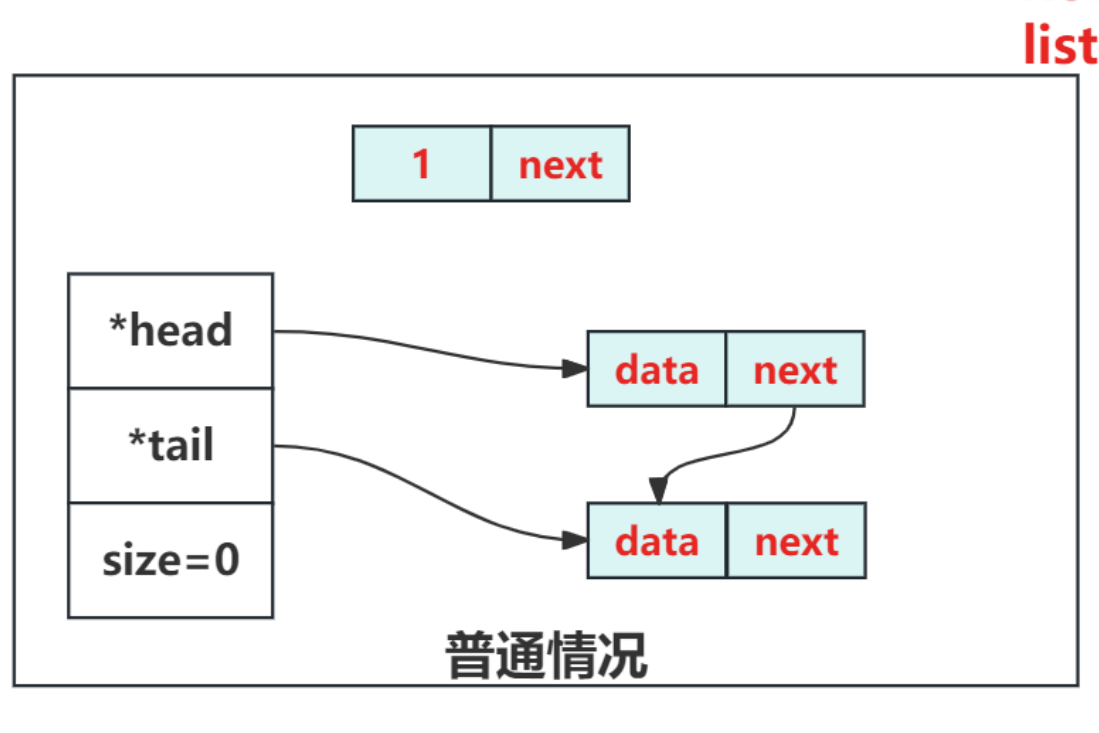
头插法


1
next
new_node

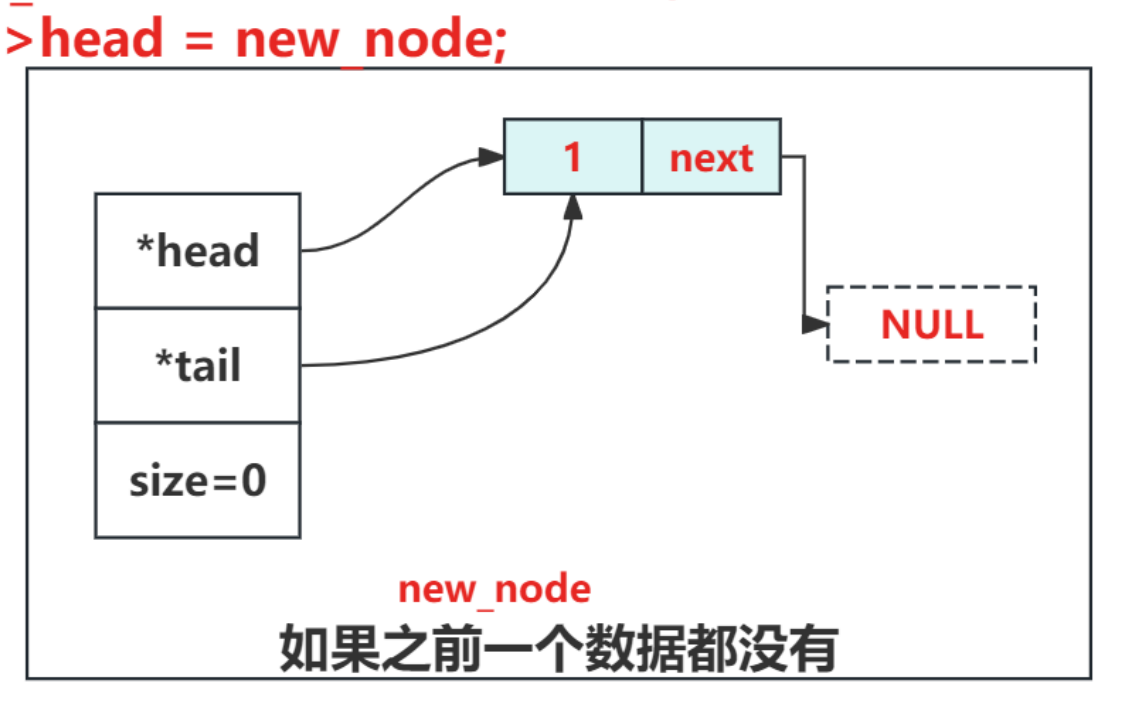
Node *new_node = malloc(sizeof(Node));

new_node->next = list->head;

new_node->data = data;
new_node->next = list->head;
list->head = new_node;

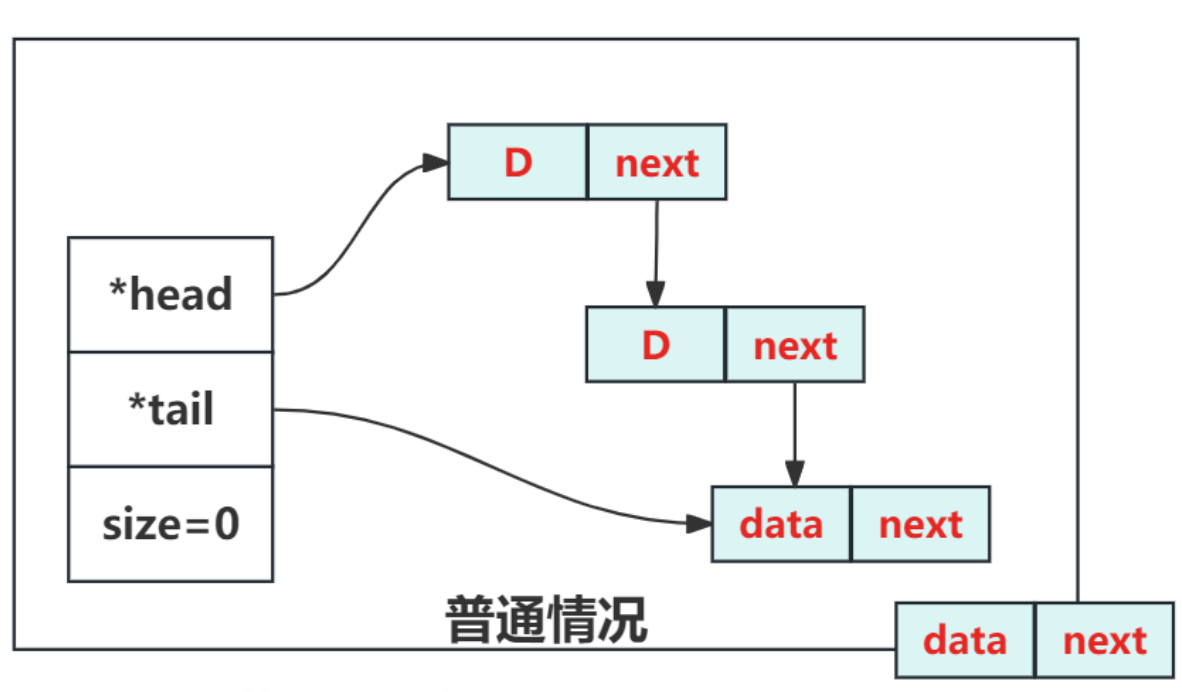

1
next
*head
*tail
size=0
data
next

普通情况

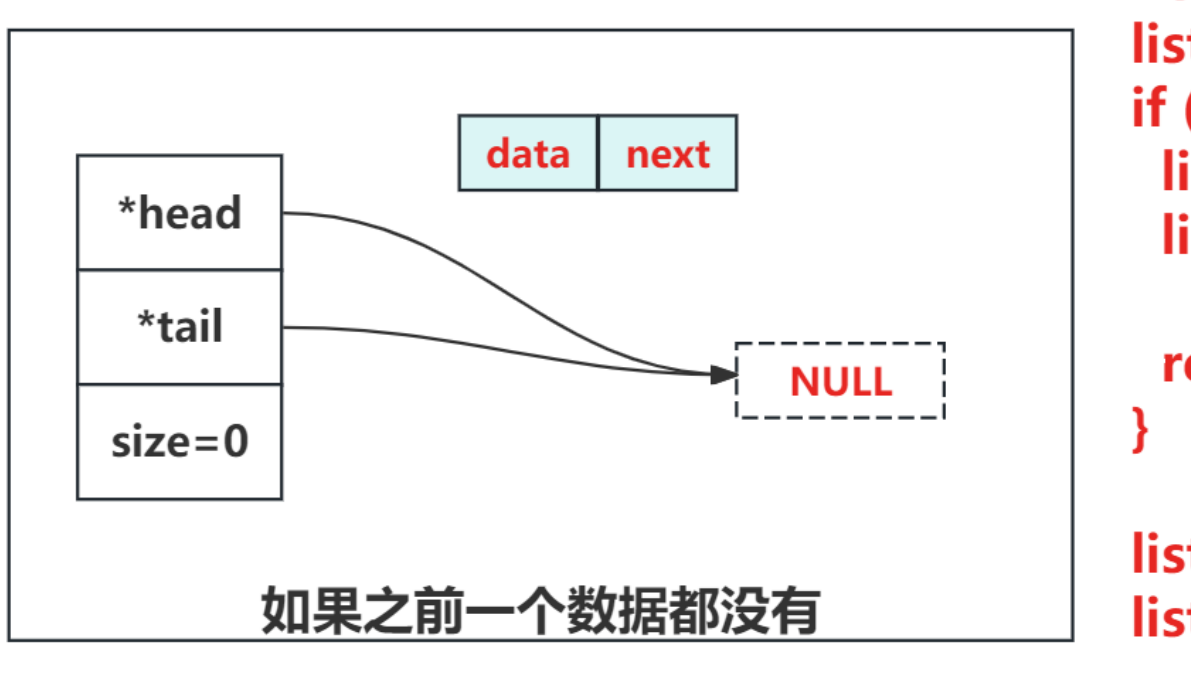

*head
*tail
size=0
1
next
NULL
new_node

如果之前一个数据都没有

尾插法


*head
*tail
size=0
D
next
data
next

普通情况


*head
*tail
size=0
data
next
NULL

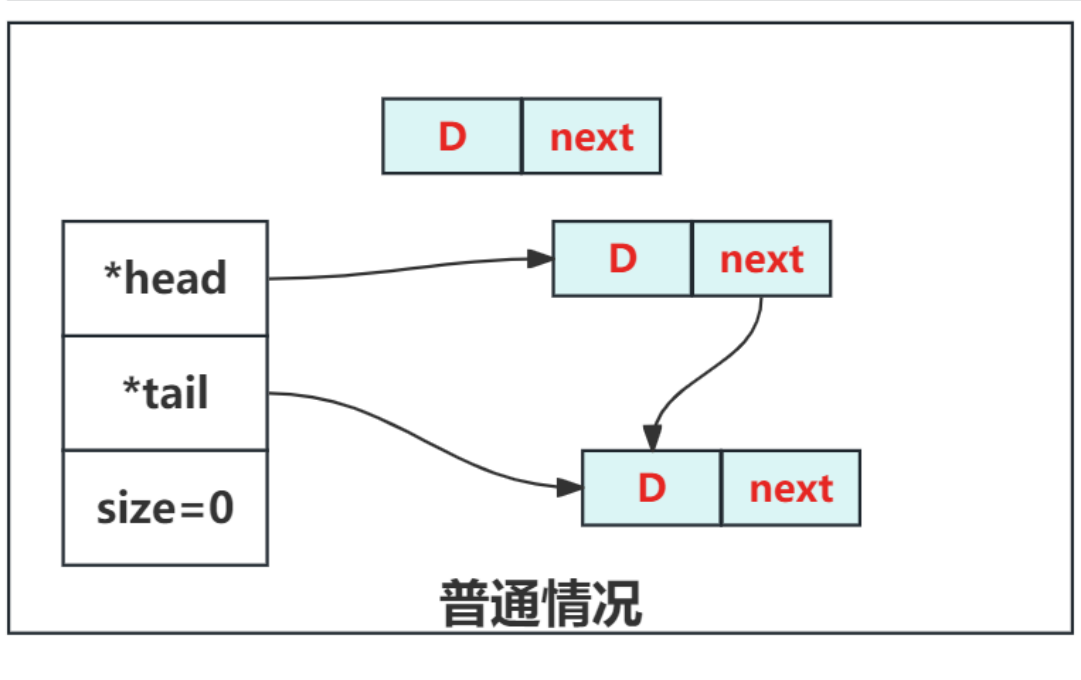
如果之前一个数据都没有

```
new_node->data = data;
list->size++;
if (list->tail == NULL){
  list->tail = new_node;
  list->head = new_node;

  return;
}

list->tail->next = new_node;
list->tail = new_node;
```

```
list->tail->next = new_node;
list->tail = new_node;
```

根据索引插入结点

1.可能会越界
2.头插的方式与普通的不太一样(index=0)，尾插也是


D
next
*head
*tail
size=0

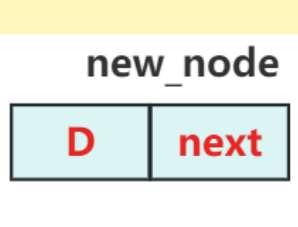
普通情况

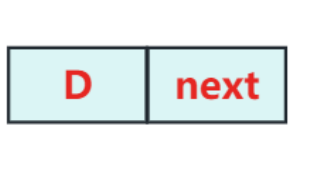
首先考虑越界，有可能传入idx=4,但是超过了我的长度。这种case，直接拒绝。

比如，链表只有三个数据，但是想往index=6插入，无法完成。

可以思考一下，允许插入的范围是?

[0, size]

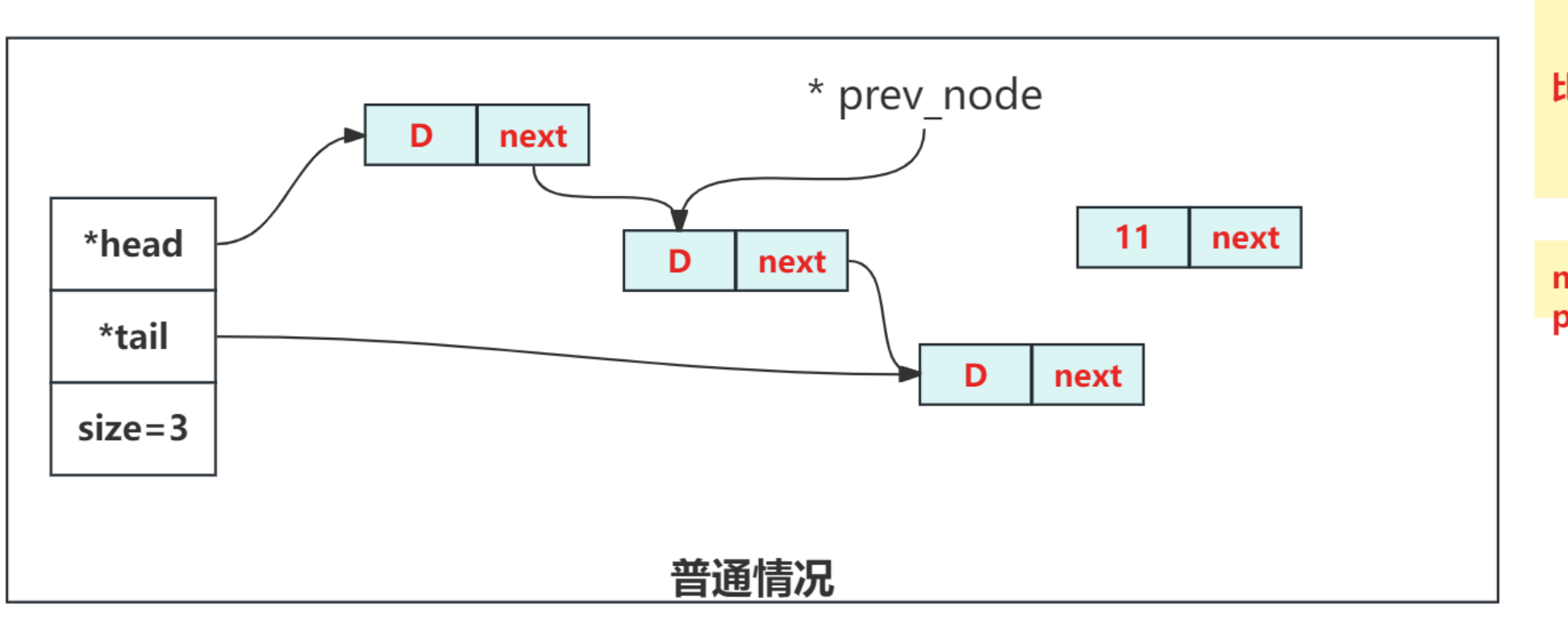

new_node
D
next



Node *cur=list->head
Node *prev = NULL;

比如，idx=2，表示想往index=2插入

比较难的，就在怎么找前置节点。

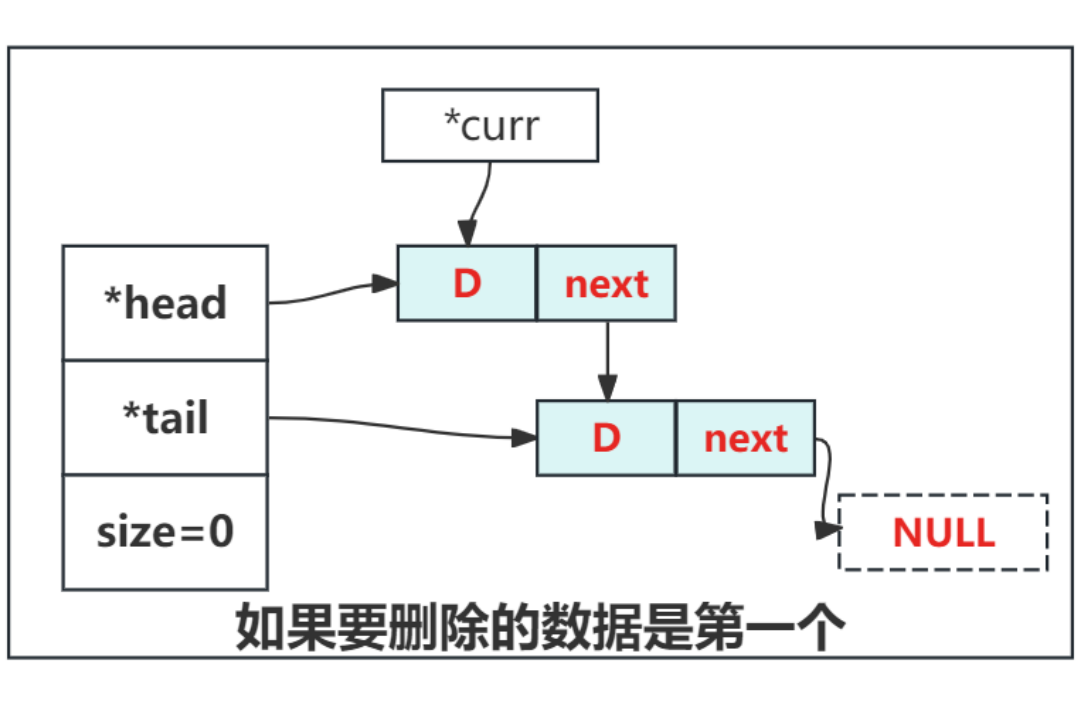

* prev_node
D
next
11
next
*head
*tail
size=3

普通情况

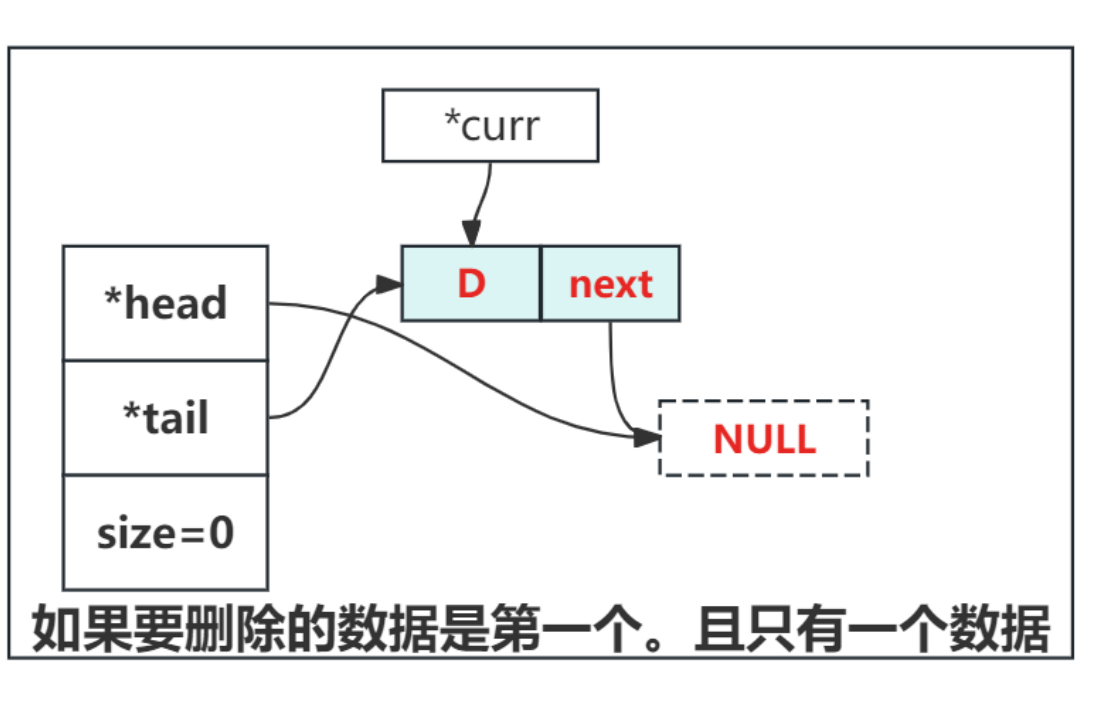
```
new_node->next = prev_node->next;
pre_node->next = new_node;
```

根据数据删除节点

先判断一下是否为空


*curr
*head
*tail
size=0
D
next
NULL

如果要删除的数据是第一个

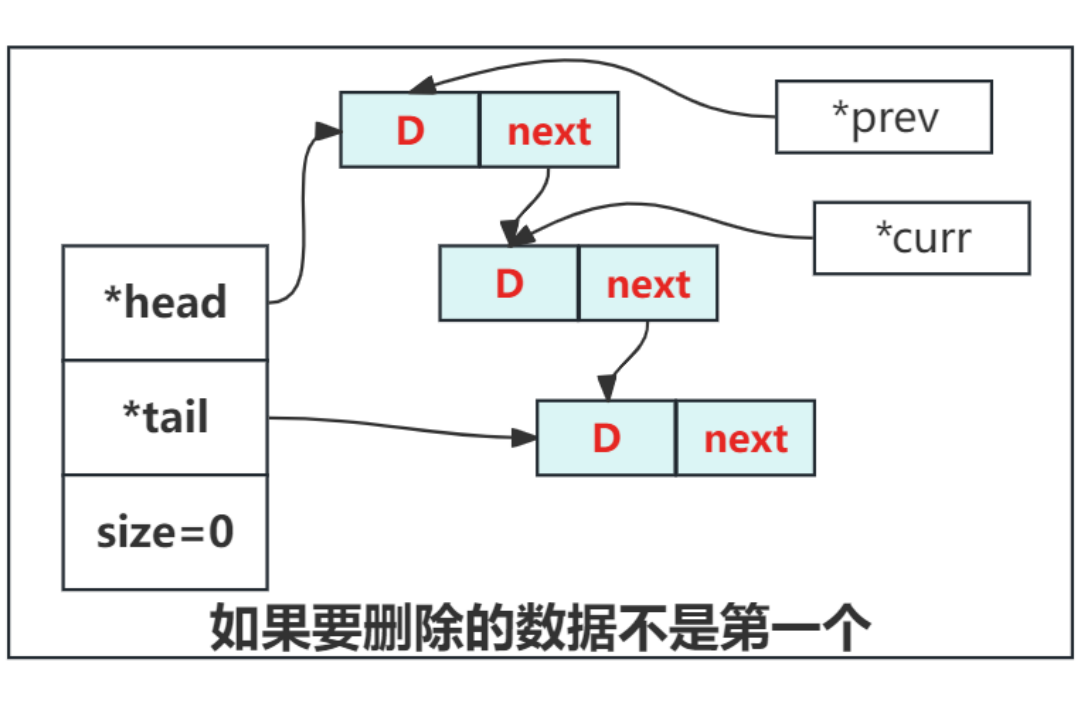

*curr
*head
*tail
size=0
D
next
NULL

如果要删除的数据是第一个。且只有一个数据

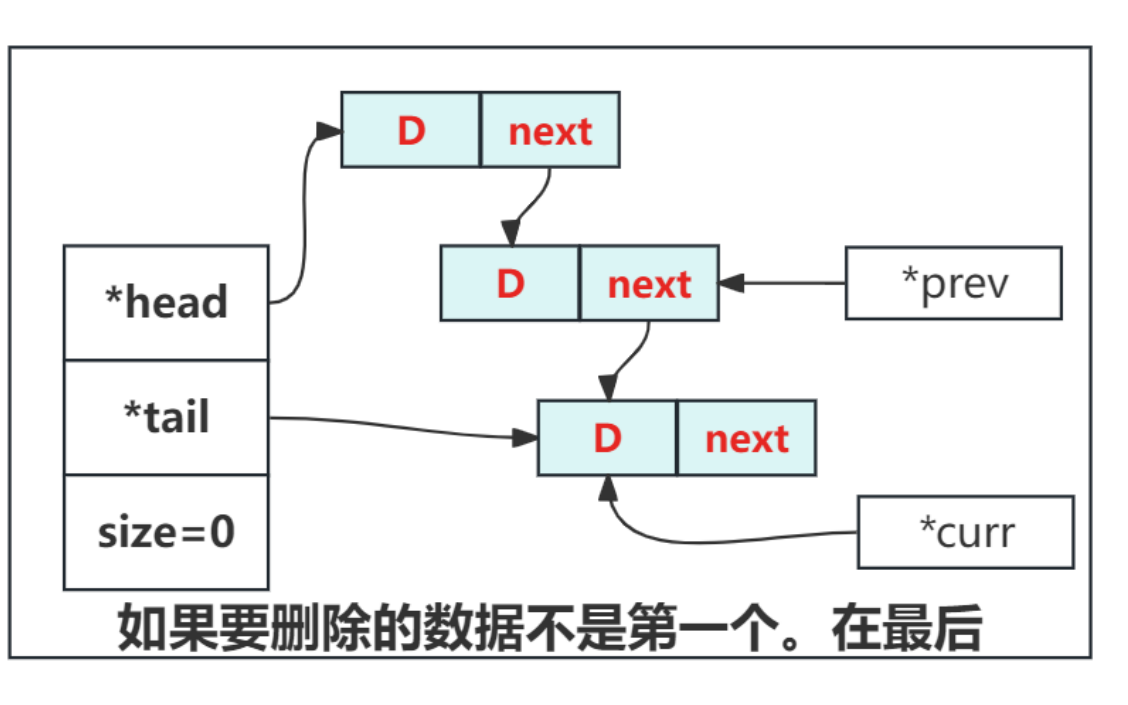
如果要删除的数据是第一个，则直接删除，并挪动。

1.为什么要单独处理，因为第一个和在中间有一些区别.

且在这个过程中，如果是第一个数据，并且只有一个数据，还要操作tail。不然会出问题


*prev
*curr
*head
*tail
size=0
D
next

如果要删除的数据不是第一个


*head
*tail
size=0
D
next
*prev
*curr

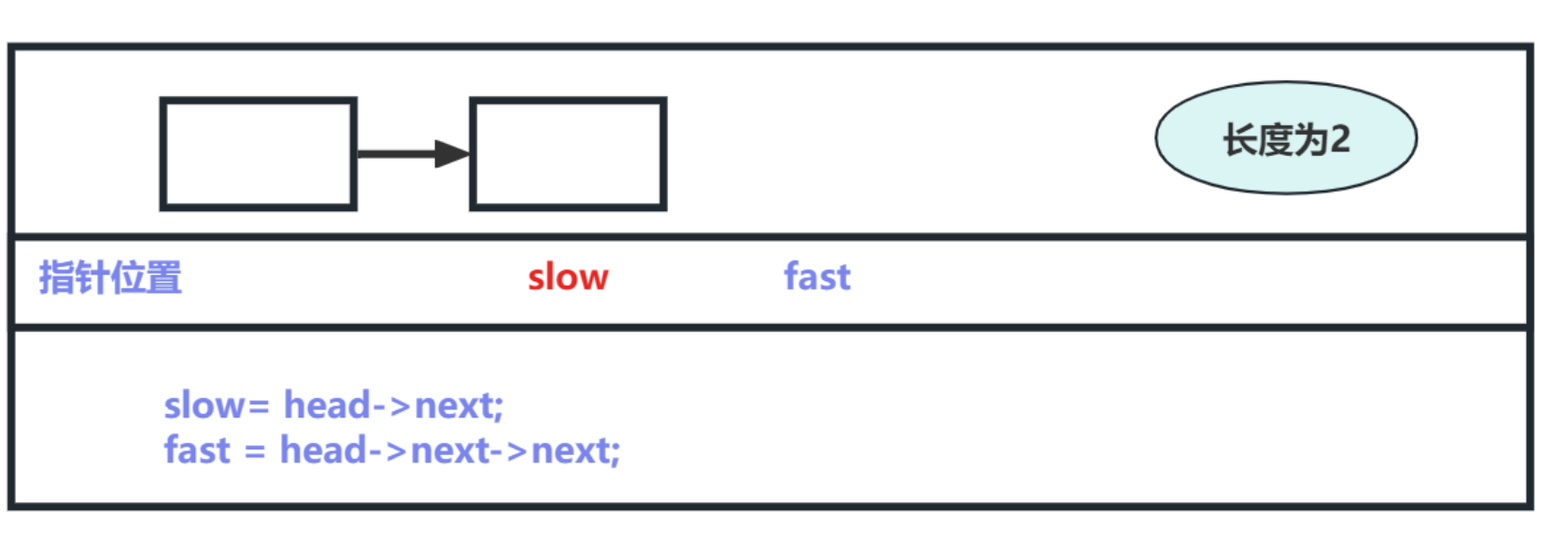
如果要删除的数据不是第一个。在最后

如果要删除的数据不是第一个。 则要找到它的前置节点。

为什么要找前驱节点？因为要删除当前节点，你只找到当前节点，这时候无法删除

如果数据是最后一个，需要处理一下尾。不然会指向一个野指针

快慢指针求中间值

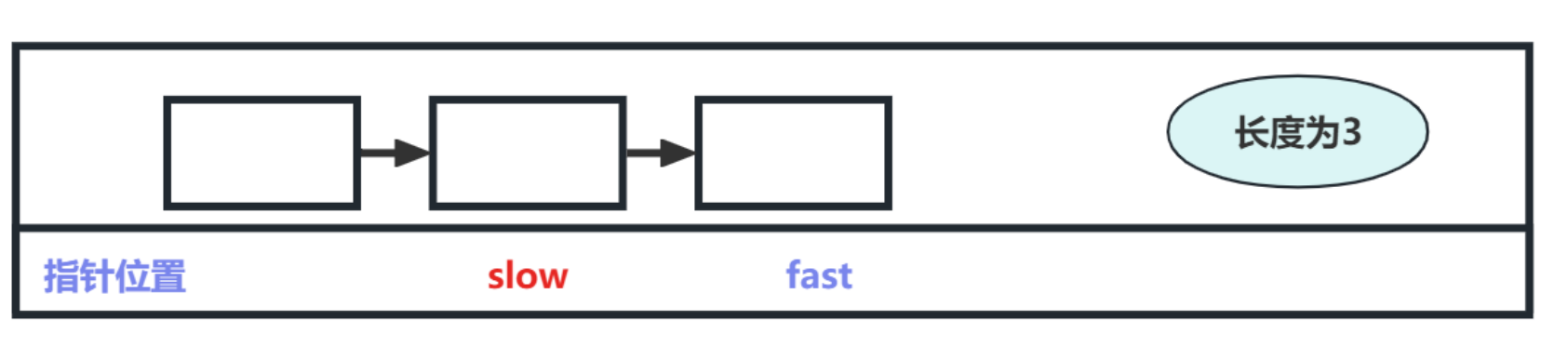

长度为2
指针位置
slow
fast

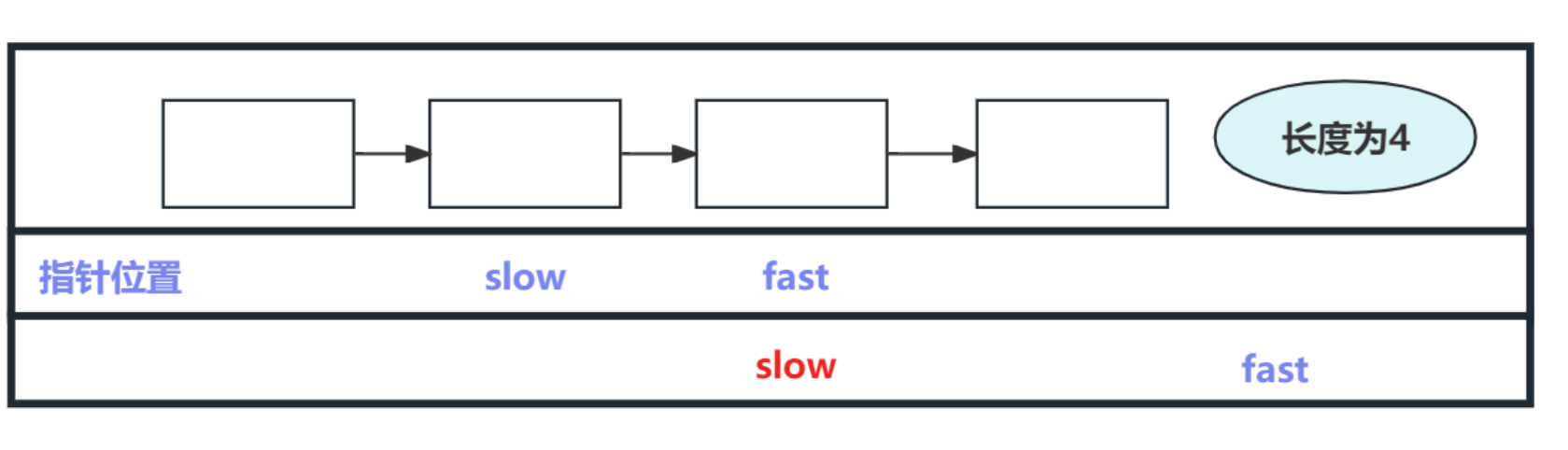
slow= head->next;
fast = head->next->next;

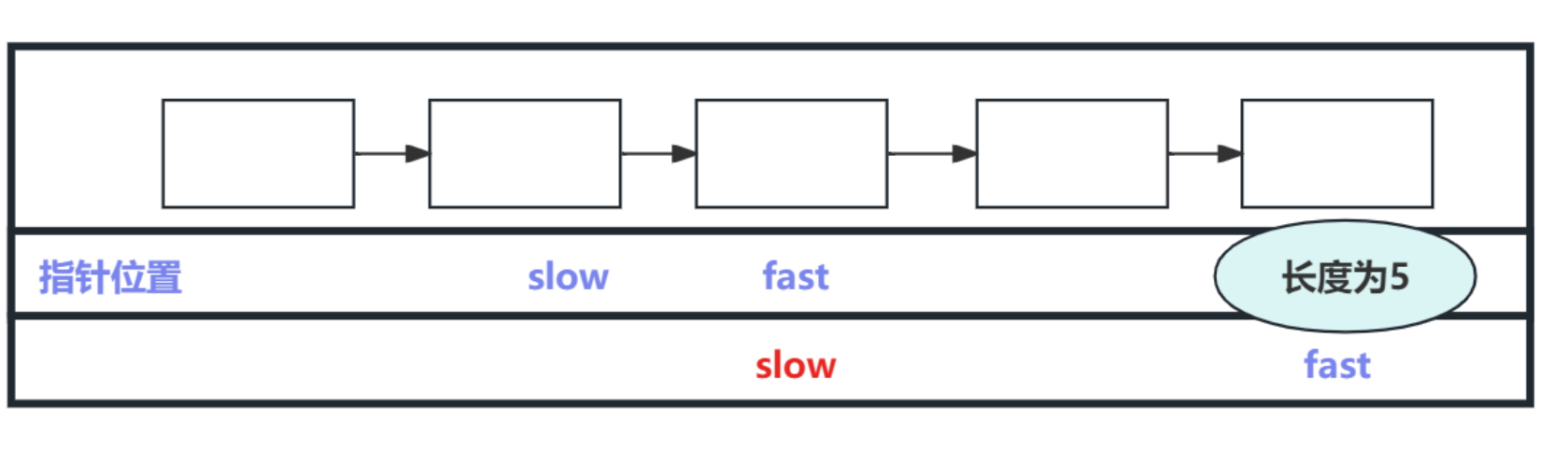
1.链表为空
直接结束

2.或者链表长度为1
直接返回第一个数据

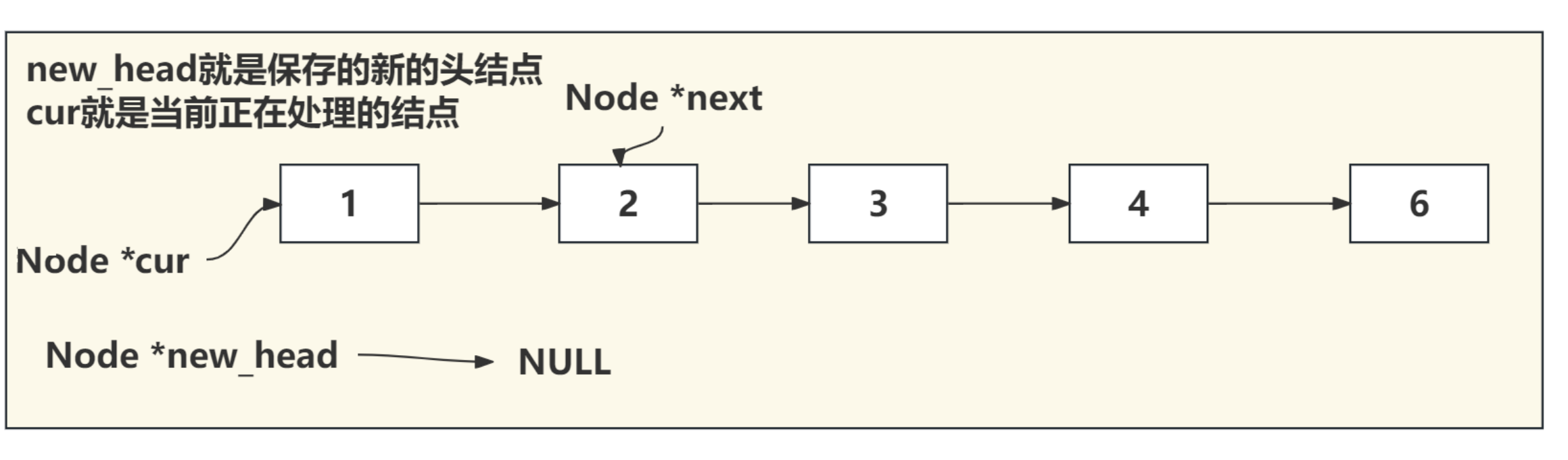
特殊情况

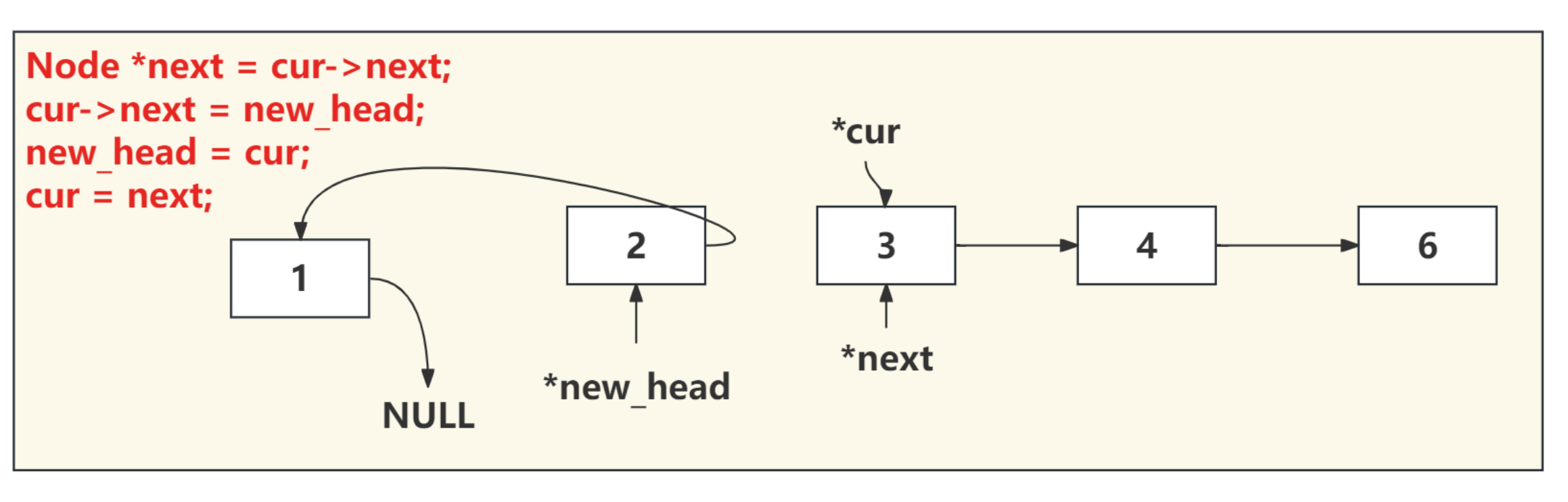

长度为3
指针位置
slow
fast


长度为4
指针位置
slow
fast
slow
fast

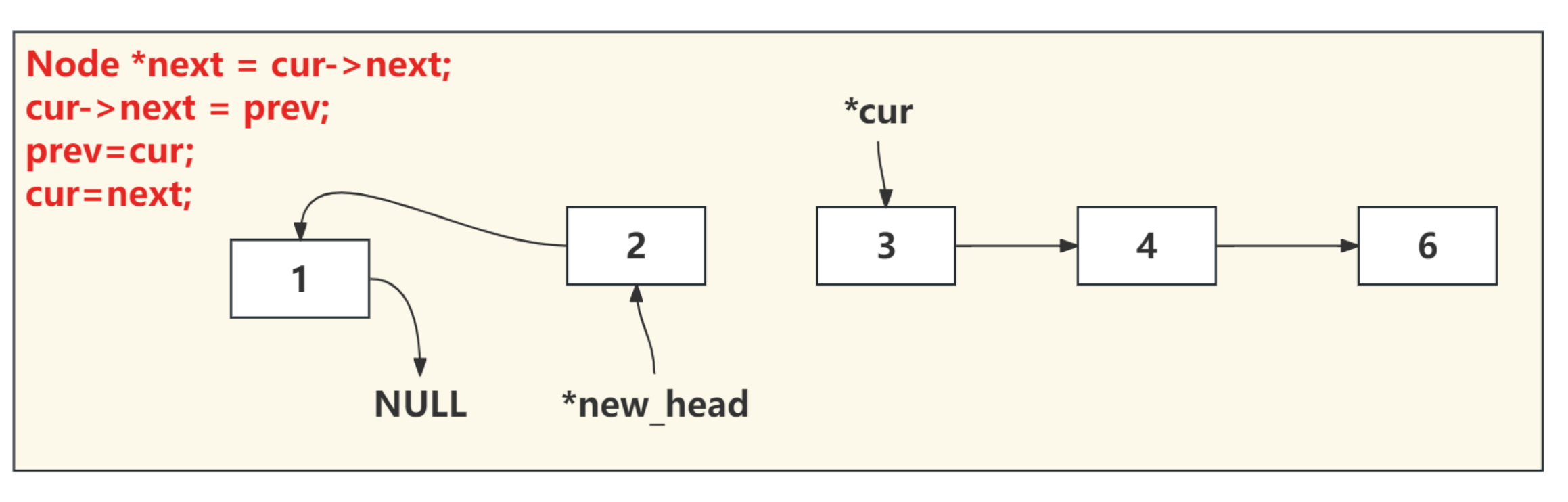

指针位置
slow
fast
长度为5
slow
fast

单向链表反转

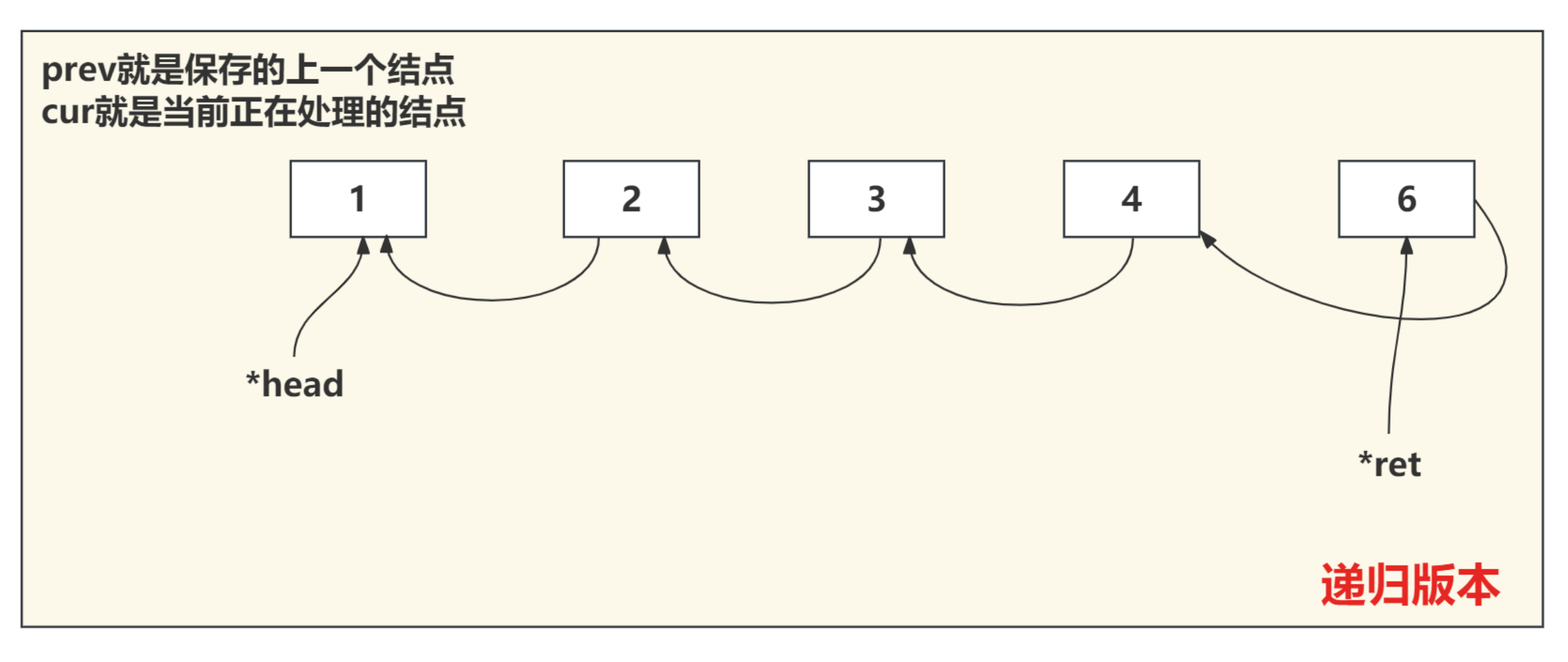

new_head就是保存的新的头结点
cur就是当前正在处理的结点
Node *next
1
2
3
4
6
Node *cur
Node *new_head
NULL


*cur
1
2
3
4
6
NULL
*new_head
*next

```
Node *next = cur->next;
cur->next = new_head;
new_head = cur;
cur = next;
```


*cur
1
2
3
4
6
NULL
*new_head

```
Node *next = cur->next;
cur->next = prev;
prev=cur;
cur=next;
```


prev就是保存的上一个结点
cur就是当前正在处理的结点
1
2
3
4
6
*head
*ret

递归版本